A tutorial on analyzing delay equations with MatCont

Davide Liessi, Enrico Santi, Rossana Vermiglio,
Mayank Thakur, Hil G. E. Meijer, Francesca Scarabel

3 February 2026

1 Introduction

In this tutorial we demonstrate the functionalities of the new Delay Equation Importer introduced in MatCont version 7p6. We assume that the reader is already familiar with MatCont for the analysis of ODEs, in particular with tutorials `lab1.pdf`, `lab2.pdf`, `lab3.pdf` and `lab4.pdf` available at https://sourceforge.net/projects/matcont/files/Documentation/MatContODE/Tutorials7px/. A tutorial on the analysis of the well-known Mackey–Glass delay differential equation is presented in [7]; for further details on the method and its implementation, we refer to that publication.

2 A renewal equation with quadratic nonlinearity

We perform a numerical bifurcation study of the renewal equation

$$x(t) = \frac{\gamma}{2} \int_{-3}^{-1} x(t+\theta)(1 - x(t+\theta))d\theta. \tag{1}$$

To import this model into MatCont, from the main MatCont GUI window we select Select > System > Delay Equation Importer > New. We define the system as

```
x[t]=gamma/2*DE_int(@(theta)x[t+theta]*(1-x[t+theta]),-3,-1)
```

with `x` as the list of coordinates and `gamma` as the list of parameters, and we choose a name for the system (e.g. `RE_quadratic`).

We keep the default collocation degree (10) and save the system by pressing OK, which generates the corresponding *odefile* `RE_quadratic.m` in the `Systems` folder and loads the system into MatCont.

We start initially from the equilibrium $x = 0$. From the main window, we select Type > Initial Point > Equilibrium. The main window automatically selects Equilibrium (EP) as the curve type, and init_EP_EP as the initializer. In the Starter window, we select γ as active parameter and set its initial value to $\gamma = 0$. The equilibrium is already initialized

to zero, so we do not need to change the values here. We keep the default parameters in the Continuer window.

To visualize the results we open a 2D plot via Window/Output > Graphic > 2D plot. To visualize the model variable x in the vertical axis, we choose MatCont > Layout, select Reconstructed coordinates (renewal equations) in the Ordinate box and press OK. We also adjust the plotting style to distinguish stable from unstable segments: for example, we can change EP if unstable and LC if unstable to `'Color', 'magenta', 'linestyle', '-'` in MatCont > Plot Properties.

We are now ready to start the continuation. From the MatCont GUI, we run Compute > Forward. The continuation detects a transcritical bifurcation (branching point, BP) at $\gamma = 1$. We can adapt the axes range in the graphical window using MatCont > Fit Range) or MatCont > Layout.

To switch to the nontrivial equilibrium branch, we open the Data Browser window via View Result or Select > Initial Point. We select the BP, press Select Point, and notice that the initial point type is now Branch Point (BP), and the initializer init_BP_EP.

We now run Compute > Forward, detecting a Hopf bifurcation at $\gamma \approx 3.5708$, approximating the analytical value $\gamma = 2 + \frac{\pi}{2}$. From the Data Browser we can select this Hopf point and keep the default curve type Limit cycle (LC) to start the continuation of the branch of periodic orbits emerging from Hopf. A new continuation parameter, the period, appears (active by default). To speed up the computation, we increase the MaxStepsize option in the Continuer to 0.5. Running Compute > Forward performs the continuation of the limit cycle branch. A period doubling (PD) is found at $\gamma \approx 4.3242$ and we stop the continuation there. We can continue the double-period limit cycle branch by selecting the PD in the Data Browser window, keeping the default initializer Limit cycle (init_PD_LC2) and Compute > Forward. A second PD on this branch is detected at $\gamma \approx 4.4959$. We perform one more step by selecting this PD as initial point and Compute > Forward. On this branch, a PD is detected at $\gamma \approx 4.5339$.

These results agree with [2, 3, 5] to which we refer for more details and other properties of (1).

On an Apple MacBook Air (M1, 2020) with 8 GB of RAM running macOS Sequoia 15.7.3 and MATLAB R2025b, the continuations of the equilibrium branches (300 points) took a few seconds and the detection of each period doubling took about a minute.

Figure 1 shows the bifurcation diagram resulting from the experiment described in this section.

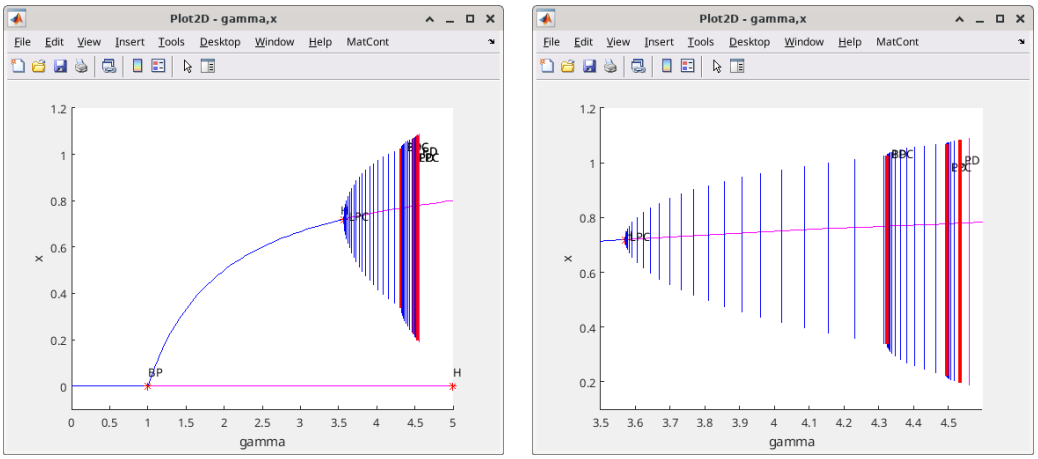

FIGURE 1. Bifurcation diagram for the RE with quadratic nonlinearity (1) with respect to γ (left) and a zoom on the part related to limit cycles, to better appreciate the period doubling bifurcations highlighted in red (right).

3 A simplified logistic Daphnia model

We analyze the system of coupled RE and DDE

$$\begin{cases} b(t) = \beta S(t) \displaystyle\int_{a_{\text{repr}}}^{a_{\max}} b(t-a)\,\mathrm{d}a \\ S'(t) = rS(t)\left(1 - \dfrac{S(t)}{K}\right) - \gamma S(t) \displaystyle\int_{a_{\text{repr}}}^{a_{\max}} b(t-a)\,\mathrm{d}a \end{cases} \tag{2}$$

where $b(t)$ denotes the density of newborns, $S(t)$ the algal concentration, and the integrals encode the reproduction and grazing rates through distributed age kernels. For more details, see [1, 4].

To import this model into MatCont, we open the Delay Equation Importer from the main MatCont GUI window via Select > System > Delay Equation Importer > New. Since the system includes two distributed delays, we specify the integration bounds as additional parameters and enter the model in the form

```
S_int_b=S[t]*DE_int(@(a)b[t-a],a_repr,a_max)
b[t]=beta*S_int_b
S'[t]=r*S[t]*(1-S[t]/K)-gamma*S_int_b
```

with `b,S` as the list of coordinates and `a_repr,a_max,beta,r,K,gamma` as the list of parameters, and we choose a name for the system (e.g. `Daphnia`). Note that we stored the value of the repeated integral in a variable for efficiency. We keep the default collocation degree (10) and accept the importer settings by pressing OK, which generates the corresponding *odefile* `Daphnia.m` in the `Systems` folder and loads it into MatCont.

We construct a bifurcation diagram with respect to β starting from the consumer-free equilibrium $(0, K) = (0, 1)$. From the main window, we select Type > Initial Point > Equilibrium. The main window automatically selects Equilibrium (EP) as the curve type, and init_EP_EP as the initializer. In the Starter window, we first set the parameter values to $a_{\text{repr}} = 3$, $a_{\max} = 4$ and $r = K = \gamma = 1$, and select β as active parameter, with initial value $\beta = 0.1$. We then set up the initial point using the option Setup initial point from function: we input $b = 0$ and $S = 1$, and press OK. We keep the default parameters in the Continuer window.

To visualize the results we first open a 2D plot via Window/Output > Graphic > 2D plot, which by default shows β as abscissa and S as ordinate. We change the vertical axis to show b through MatCont > Layout and selecting Reconstructed coordinates (renewal equations). We also adjust the plotting style via MatCont > Plot Properties to distinguish stable from unstable segments (by editing EP if unstable and LC if unstable as in Section 2).

We are now ready to start the continuation. From the MatCont GUI, we run Compute > Forward. The continuation detects a branching point at $\beta = 1$, and a couple of Hopf points on the unstable branch (we can adapt the axes range in the graphical window using MatCont > Fit Range) or MatCont > Layout). We stop the continuation and prepare to compute the nontrivial equilibrium branch.

To select the branching point, we open the Data Browser window via View Result or Select > Initial Point. We select the BP, press Select Point, and notice that the initial point type is now Branch Point (BP), and the initializer init_BP_EP.

We now run Compute > Forward, detecting a Hopf bifurcation at $\beta \approx 3.0161$. From the Data Browser we can select this Hopf point and leave the curve type as Limit cycle (LC) to start the continuation of the branch of periodic orbits emerging from Hopf. A new continuation parameter, the period, appears (active by default). To speed up the computation, we increase the MaxStepsize option in the Continuer to 1; we also set MaxNumPoints to 100. Running Compute > Forward completes the continuation of the limit cycle branch. No bifurcation points were detected on this branch.

On an Apple MacBook Air (M1, 2020) with 8 GB of RAM running macOS Sequoia 15.7.3 and MATLAB R2025b, the continuations of the equilibrium branches (300 points each) took less than 10 seconds, while the continuation of the limit cycle branch (100 points) took considerably more time, approximately 110 seconds.

As next test, we perform a two-parameter continuation of the Hopf curve in the plane (β, K). To do this, we select Type > Curve > Hopf (init_H_H). We clear the graphical window with MatCont > Clear and change the layout via MatCont > Layout to show the parameters β and K in the abscissa and ordinate axes, respectively. Then, we go back to the default continuation parameter 0.1 for MaxStepsize, while we set MaxNumPoints to 150. We select β and K as active parameters in the Starter window and, in the MatCont GUI, Compute > Forward and Compute > Backward.

Figure 2 shows the bifurcation diagrams resulting from the experiments described in this section.

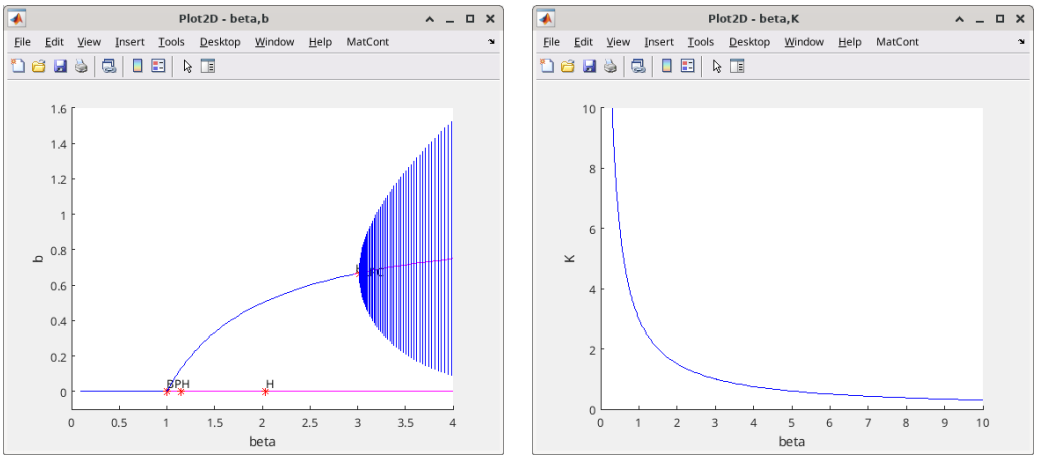

FIGURE 2. Bifurcation diagrams for the simplified logistic Daphnia model (2): continuation of b with respect to β (left) and Hopf curve in the plane (β, K) (right).

4 A refractory neural network model

Consider the DDE [6]

$$u'(t) = r\left(-u(t) + \left(1 - \int_{t-1}^{t} u(s)ds\right)\frac{1}{1+e^{-a(u(t)+\theta)}}\right).$$

To input it in MatCont, we make a change of variable in the integral to make the integration endpoints independent of t:

$$u'(t) = r\left(-u(t) + \left(1 - \int_{-1}^{0} u(t+s)ds\right)\frac{1}{1+e^{-a(u(t)+\theta)}}\right). \tag{3}$$

To define the system, we input

```
u'[t]=r*(-u[t]+(1-DE_int(@(s)u[t+s],-1,0))*1/(1+exp(-a*(u[t]+theta))))
```

with `u` as the list of coordinates and `r,a,theta` as the list of parameters.

To explore the system's dynamics, we first perform a time integration. From the main window, we select Type > Initial Point > Point, and in the Starter we set the parameters at $r = 1$, $a = 8$, $\theta = -0.333$, and set the initial history function via Setup initial point from function as the constant equal to 1. In the Integrator window, we set the integration interval: in this case 50 (corresponding to the interval $[t, t+50]$) is sufficient. We then open a 2D plot and choose t as abscissa and u as ordinate. We run Compute > Forward and note that the solution tends to the equilibrium $u \approx 0.3359$, as shown in Figure 3 (left). From the Data Browser window, we select the last point (Select Point) to start an equilibrium continuation.

To do this, in the main window we select Type > Initial Point > Equilibrium, which also automatically changes the curve type to Equilibrium (EP). In the Starter window we select

r as active parameter, and we prepare the graphic window clearing its content with MatCont > Clear and setting r as abscissa and u as ordinate from MatCont > Layout. With Compute > Forward we find a Hopf bifurcation at $r \approx 4.8394$. Starting from the Hopf point (from the Data Browser) we can compute the branch of limit cycles. We keep the default continuation parameter 0.1 for MaxStepsize and we set MaxNumPoints to 100. Figure 3 (right) shows the resulting bifurcation diagram.

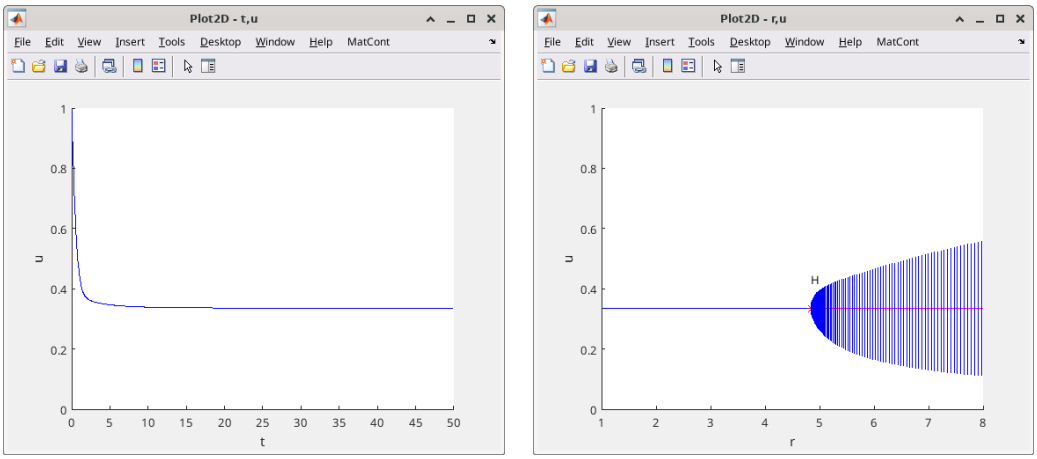

FIGURE 3. Simulation (left) and bifurcation diagrams (right) for the refractory neural model (3).

5 A two-node neural network

The system from [8] of two DDEs with two delays

$$\begin{cases} x_1'(t) = -x_1(t) - \alpha_1 S(\beta_1 x_1(t-\tau_1)) + \alpha_2 S(\beta_2 x_2(t-\tau_2)) \\ x_2'(t) = -x_2(t) - \alpha_1 S(\beta_1 x_2(t-\tau_1)) + \alpha_2 S(\beta_2 x_1(t-\tau_2)) \end{cases} \tag{4}$$

with $S(x) = (\tanh(x-a) + \tanh(a))\cosh(a)^2$ can be input as

```
S=@(x)(tanh(x-a)+tanh(a))*cosh(a)^2
x1'=-x1[t]-alpha1*S(beta1*x1[t-tau1])+alpha2*S(beta2*x2[t-tau2])
x2'=-x2[t]-alpha1*S(beta1*x2[t-tau1])+alpha2*S(beta2*x1[t-tau2])
```

with `x1,x2` as the list of coordinates and `a,tau1,tau2,alpha1,alpha2,beta1,beta2` as the list of parameters. Note that, as done for (2), we stored the definition of the function S for convenience.

We want to reproduce the bifurcation diagram with respect to α_2 presented in Figure 4 (top) from [8]. The dynamics of the model are rather rich and the complexity requires a degree of the collocation polynomials larger than the default: we choose 20. Moreover, the computational time to reproduce the whole bifurcation diagram is

quite large, taking about 200 minutes in total on an HP Probook 430 G7 with Intel Core i5-10210U CPU (1.60 GHz) and 16 GB of RAM running Fedora Linux 42 and MATLAB R2024b.

Given the large computational time and the number of special points that are detected on the branches, it is convenient to let MatCont finish the continuation of each branch without pausing at special points, so that the computation can complete unattended: we do this from the main window choosing Never in Options » Suspend Computation.

By default, MatCont keeps the last two branches of each kind, overwriting the older ones, but if a branch is renamed then it is always preserved. Branches can be renamed via Select » Curve and the Rename Curve button, or, after the computation of the current branch, by right-clicking on the Current Curve box and selecting Rename Curve.

We start from the trivial equilibrium. We select Type » Initial Point » Equilibrium. In the Starter window we set the parameters $a = 1$, $\tau_1 = 11.6$, $\tau_2 = 20.3$, $\alpha_1 = 0.069$, $\beta_1 = 2$, and $\beta_2 = 1.2$, and select α_2 as active parameter, with initial value $\alpha_2 = 0.1$. The initial point is already set to 0 and we keep the default parameters in the Continuer window. We open a 2D plot via Window/Output » Graphic » 2D plot, which by default shows α_2 as abscissa and x_1 as ordinate. We can use MatCont » Layout to set the shown intervals to $\alpha_2 \in [0.4, 1]$ and $x_1 \in [-0.5, 3]$. As in the previous examples (see Section 2), we make sure via MatCont » Plot Properties that stable from unstable segments are plotted differently. We start the continuation with Compute » Forward, discovering, among several special points, a Hopf bifurcation at $\alpha_2 \approx 0.77090$ (H_1) and a branching point at $\alpha_2 \approx 0.94833$ (B_1). The computation of this branch (300 points) takes 23 seconds. We rename this curve as `EP_trivial`.

From the Data Browser window (via View Result or Select » Initial Point) we select the branching point B_1 and start the continuation with Compute » Forward. In the plot window we see that after a few points the branch exits the graph from the bottom-right corner: when that happens (in a few seconds), we can stop the continuation. We rename this curve as `EP_nontrivial_forward`. We continue the same branch in the other direction with Compute » Backward. Among several special points, we find a limit point of equilibria, or fold, at $\alpha_2 \approx 0.52110$ (F_1). The computation (300 points) takes 17 seconds. We rename this new curve as `EP_nontrivial_backward`.

We observe that the Hopf bifurcations H_2 and H_3 shown along the branch after F_1 in [8, Figure 4 (Detail IV)] are not detected by MatCont. This is probably due to the step size being too large to detect special points so close to each other. We will get back to this later.

We now want to continue the branch of limit cycles exiting from the Hopf point H_1. From the main window we reload the curve `EP_trivial` with Select » Curve and choose H_1 as an initial point with Select » Initial Point. The curve type is correctly set to Limit cycle (LC). In the interest of saving computation time, in the Starter window we disable the detection of Neimark–Sacker (NS) bifurcations, since they were not detected in [8]. We also increase the MaxStepsize option in the Continuer to 1 and we set MaxNumPoints to

360. We select Compute > Forward and find something else to do while the continuation is running: in fact, the computation of this branch took about 130 minutes. We rename this curve as `LC_from_H1`. Along the branch, several special points are detected, among which we find the ones listed in Table 1, which we can compare with [8, Table 1].

When we return to MatCont, we find the message `Current step size too small` in MATLAB's Command Window. We observe that the last point of the branch corresponds to $\alpha_2 \approx 0.52122$ and $x_1 \approx 1.36762$, which is the point H_2 in [8, Figure 4 (Detail IV)]. The default plot setting for limit cycles (plotting the whole orbit) has made the graph too crowded to appreciate what is happening, so we want to redraw the curves to make them more readable. Unfortunately, at the moment the other available plot settings (maximum or minimum of the orbit) are not compatible with equilibria, so switching mode will delete the existing curves. A workaround is to first plot the limit cycle curves in an alternative setting and then switch back to plotting equilibria in the default mode. We clear the plot window with MatCont > Clear, open MatCont > Layout, choose the Max option for the ordinate and select MatCont > Redraw Curve; we then load the `EP_nontrivial_backward` curve, open MatCont > Layout, choose x_1 as the ordinate and select again MatCont > Redraw Curve. Now it is clear that the branch of limit cycles is ending on the branch of nontrivial equilibria, and zooming in we note that the contact point is beyond F_1. Even without the information of [8], this suggests that the contact point may be an undetected Hopf bifurcation, that we now want to find.

From Select > Initial point we choose the limit point (LP) corresponding to F_1. The curve type is auromatically set to Limit point (LP), so we need to switch it back to Equilibrium (EP) via Type > Curve. Since the maximum step size 0.1 used for the equilibrium branch `EP_nontrivial_backwards` was too large, we set MaxStepsize in the Continuer to 0.001, and we set MaxNumPoints to 100. Indeed, with Compute > Forward we recover the points H_2 and H_3 mentioned above and listed in Table 1, and we observe that the former is the endpoint of the branch of limit cycles we computed previously. The computation took a few seconds. We rename this curve as `EP_nontrivial_from_F1`.

Now that we also have H_3, we want to continue the branch of limit cycles exiting from there. We want to plot the branch with the Max option, but selecting it clears the plot window and we want to keep the other limit cycle branch as reference. We load the `LC_from_H1` curve, clear the plot window with MatCont > Clear, open MatCont > Layout, choose the Max option for the ordinate and select MatCont > Redraw Curve. We then open Select > Curve, double-click on the `EP_nontrivial_from_F1` curve again, select H_3 and click on Select Point; the Max plot option is preserved. We set MaxStepsize to 1 and MaxNumPoints to 500, check that the detection of NS bifurcations is disabled in the Starter window, and hit Compute > Forward. Again, the computation is very long, and took about 40 minutes. We rename this curve as `LC_from_H3`. We can observe that the continuation reaches a limit point of cycles in correspondence with PD_1 and then bounces back along the same branch. Significant points for comparison with [8, Table 1] are listed in Table 1. Note that there are two candidate points corresponding to LPC_4:

the maximum of x_1 along the limit cycle at LPC_{4a} is 2.59044, while the one at LPC_{4b} is 2.60215.

In the last part of this experiment, we continue the branch of double-period limit cycles starting from PD_4. We load the `LC_from_H3` curve and select the point corresponding to PD_4 from Select > Initial Point. We keep MaxStepsize at 1, set MaxNumPoints to 280, and check that the detection of NS bifurcations is still disabled in the Starter window. In our experience, starting this continuation is difficult. Apparently the basin of attraction of the double-period cycle is very small and the initialization function often fails to switch branch and falls back on two periods of the single-period cycle. One can notice when this happens because the new curve perfectly overlaps the previous one. The initialization step involves random perturbations, so repeating it multiple times eventually succeeds in switching to the correct branch. The computation took about 30 minutes. We rename this curve as `LC_from_PD4`. A limit point of cycles is detected for $\alpha_2 \approx 0.61795$ and corresponds to the one shown (but not highlighted) in [8, Figure 4 (Detail II)]. Another limit point of cycles is detected for $\alpha_2 \approx 0.55740$, after which the curve bounces back along the same branch, suggesting that it corresponds to the endpoint of the branch; however, this does not correspond to [8, Figure 4 (Detail I)], which shows this branch ending at PD_3 and further more refined analysis would be needed.

Branch	Point	$\alpha_2 \approx$	Type
`EP_trivial`	H_1	0.77090	Hopf
	B_1	0.94833	branching point
`EP_nontrivial_forward`	F_1	0.52110	limit point (fold)
`LC_from_H1`	PD_1	0.65035	period doubling
	LPC_1	0.46197	limit point of cycles
	PD_2	0.46483	period doubling
	PD_3	0.59893	period doubling
	LPC_2	0.61763	limit point of cycles
	PD_4	0.52174	period doubling
`EP_nontrivial_from_F1`	H_2	0.52120	Hopf
	H_3	0.52127	Hopf
`LC_from_H3`	LPC_3	0.46374	limit point of cycles
	LPC_{4a}	0.61949	limit point of cycles
	LPC_{4b}	0.61941	limit point of cycles

TABLE 1. Significant special points detected in the bifurcation analysis of (4).

Finally, we construct the bifurcation diagram shown in Figure 4. As we have already noted, choosing the Max plot option clears the curves plotted in the default mode. We

can proceed as before, first plotting all the limit cycle curves in Max and then plotting the equilibrium curves in default mode. We clear the figure with MatCont > Clear, choose the Max option in MatCont > Layout and select MatCont > Redraw Diagram: in fact, equilibrium curves, being incompatible with the selected plot option, are skipped. Then, one by one, we load each equilibrium curve, check the settings in MatCont > Layout and select MatCont > RedrawCurve.

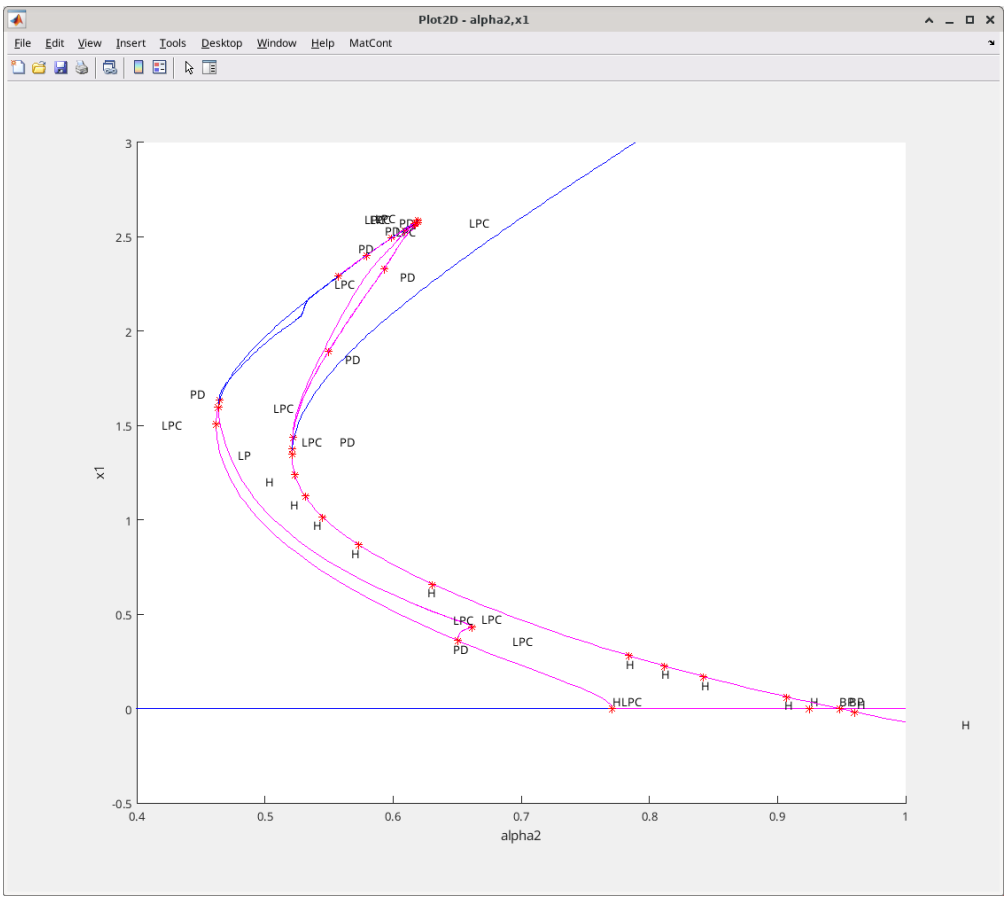

FIGURE 4. Bifurcation diagram for the two-node neural network (4).

More dynamical properties of (4) can be explored and we refer to [8] for more details to inform such investigation.

References

[1] D. BREDA, O. DIEKMANN, M. GYLLENBERG, F. SCARABEL, AND R. VERMIGLIO, *Pseudospectral discretization of nonlinear delay equations: new prospects for numerical bifurcation analysis*, SIAM J. Appl. Dyn. Syst., 15:1 (2016), pp. 1–23, DOI: 10.1137/15M1040931 (cit. on p. 3).

[2] D. Breda, O. Diekmann, D. Liessi, and F. Scarabel, *Numerical bifurcation analysis of a class of nonlinear renewal equations*, Electron. J. Qual. Theory Differ. Equ., 2016, 65 (2016), pp. 1–24, doi: 10.14232/ejqtde.2016.1.65 (cit. on p. 2).

[3] D. Breda and D. Liessi, *Approximation of eigenvalues of evolution operators for linear renewal equations*, SIAM J. Numer. Anal., 56:3 (2018), pp. 1456–1481, doi: 10.1137/17M1140534 (cit. on p. 2).

[4] D. Breda and D. Liessi, *Approximation of eigenvalues of evolution operators for linear coupled renewal and retarded functional differential equations*, Ric. Mat., 69:2 (2020), pp. 457–481, doi: 10.1007/s11587-020-00513-9 (cit. on p. 3).

[5] D. Breda and D. Liessi, *A practical approach to computing Lyapunov exponents of renewal and delay equations*, Math. Biosci. Eng., 21:1 (2024), pp. 1249–1269, doi: 10.3934/mbe.2024053 (cit. on p. 2).

[6] R. Curtu and B. Ermentrout, *Oscillations in a refractory neural net*, J. Math. Biol., 43 (2001), pp. 81–100, doi: 10.1007/s002850100089 (cit. on p. 5).

[7] D. Liessi, E. Santi, R. Vermiglio, M. Thakur, H. G. E. Meijer, and F. Scarabel, *New functionalities in MatCont: delay equations and lyapunov exponents*, arXiv: 2504.12785 [math.DS], submitted (cit. on p. 1).

[8] S. Visser, H. G. E. Meijer, M. J. A. M. van Putten, and S. A. van Gils, *Analysis of stability and bifurcations of fixed points and periodic solutions of a lumped model of neocortex with two delays*, J. Math. Neurosci., 2, 8 (2012), doi: 10.1186/2190-8567-2-8 (cit. on pp. 6–10).